

RAPID: A Generalized High-Performance Parallelization Framework for Real-Time Deep Learning Seismic Phase Picking

C. Skevofilax, V. Salles, C. Muñoz, A. Savvaidis
University of Texas at Austin

May 4, 2026

Abstract

Deep learning has transformed seismic phase picking, yet deploying these models at network scale in real time remains an unresolved engineering challenge. Unified libraries such as SeisBench excel at high-throughput offline batch inference but are ill-suited for continuous monitoring, where waveforms arrive asynchronously from individual stations and cannot be buffered until a full batch is assembled. The naive alternative—dispatching each station as an independent parallel task—forces repeated framework initialization for every waveform, inflating wall times and consuming most of the available compute budget unless concurrency and hardware are carefully tuned. To resolve this, we introduce **RAPID (Resource-Aware Parallel Inference Dispatcher)**, a generalized orchestration framework implementing two task parallelization strategies: **Ripper**, an ephemeral task-based approach, and **Model-Actor**, which maintains persistent inference instances loaded in memory across an entire processing window. We benchmarked RAPID across five deep learning phase pickers (PhaseNet, PhaseNetLight, EQTransformer, EQTransformer-NonConservative, and EQCCT) using 228 three-component 60-second waveforms from the Texas Seismological Network under hardware constraints that reflect real operational deployment scenarios. Across our benchmark grid, the Model-Actor strategy achieved real-time processing, reducing total runtime by approximately **50–87%** relative to the best Ripper results at 228 stations for the same model–hardware pairings. These results demonstrate that the dominant bottleneck in network-scale deep learning picking is orchestration overhead rather than raw inference complexity, and that persistent-actor lifecycle management delivers substantially faster runtimes and larger headroom under the 30-second target than even the best-tuned ephemeral-task baseline—all while using minimal compute resources (20 CPUs).

1 Introduction

Accurate and timely identification of seismic phase arrivals is fundamental to earthquake monitoring. For decades, seismic networks have relied on energy-ratio algorithms, primarily the Short-Term Average/Long-Term Average (STA/LTA), to automate event detection. While computationally inexpensive, these algorithms are sensitive to background noise, require station-specific tuning, and often produce high false-positive rates in complex noise environments.

Over the past decade, deep learning has emerged as a powerful alternative. Models such as PhaseNet (Zhu & Beroza, 2018), EQTransformer (Mousavi et al., 2020), and EQCCT (Saad et al., 2023) consistently outperform traditional picking methods in both precision and recall, particularly in low signal-to-noise conditions. SeisBench (Woollam et al., 2022) has consolidated many of these models into a unified interface along with several datasets (Münchmeyer et al., 2022), establishing itself as the standard library for deep learning–based seismic phase-picking research.

This progress has naturally led researchers to incorporate these pickers into production workflows with real-time operations in mind. We define *real-time processing* as returning picks for a full station selection within 30 seconds of a 60-second processing window. Chen, Savvaidis, et al. (2024) describe a near real-time workflow using EQCCT in SeisComP as the main picking algorithm, together with association, relocation, and catalog quality control. Their evaluation emphasizes catalog quality and analyst workflow rather than strict network-scale bounds on picking latency alone. Yeck et al. (2020) describe a related deployment at NEIC using compact CNNs that refine STA/LTA detections before association. Their reported timings use replayed automatic product streams, which like our offline waveform benchmark, characterizes pipeline throughput but does not address how large-window deep learning pickers behave when hundreds of stations must finish within a strict local compute budget. Sheen et al. (2023) replace traditional (non-deep learning) single-channel Earthworm pickers with a module that walks 30-second windows every second on WAVE_RING using ordinary Earthworm utilities—fast for that ring-based path but still focused on classical detectors rather than coordinating regional deep learning pickers over many stations at once.

These systems are instructive, yet none solves our core question: **how to run today’s most widely used deep learning pickers when restrictive CPU, GPU, memory, and timing constraints are imposed**. We therefore started with SeisBench, where many of the most widely used models are already accessible, and asked what solutions on the platform enable real-time processing once station count and the 30-second window are jointly constrained by hardware limitations.

Currently, SeisBench supports two primary waveform processing modes. The first, *offline batch mode*, processes waveforms in a single `annotate()` call. This method uses internal batching to achieve sub-second runtimes for entire networks, processing 228 stations in about 0.24–0.52 s on our CPU/GPU system (Table 1). While highly effective for post-event analysis, it is incompatible with real-time monitoring. Operationally, waveforms arrive asynchronously from individual stations due to network latency or system failures. Consequently, we cannot buffer waveforms for batching; the advancing processing window would force the system into a near real-time mode, which is inconsistent with our detection objectives.

The second mode, *per-station streaming*, uses the `classify()` method to process one waveform at a time. Because one model instance stays loaded, this avoids per-station model startup. However, end-to-end time still grows roughly linearly with station count. At 228 stations, a full sequential `classify()` pass spans about 1.47–27.58 s on CPU and 1.47–28.63 s on GPU across the four models (Table 1). The slowest CPU case (PhaseNet) approaches the 30 s target at 228 stations (28.63 s) and exceeds it when tasked with processing 250 stations (32.58 s). Studying the `classify()` approach revealed four structural limitations:

(1) **Hardware non-scalability**. Because `classify()` advances through stations sequentially in a single process, wall-clock throughput is governed by that sequential loop. Assigning more CPU cores or exposing additional GPUs does not shorten end-to-end time unless the implementation is refactored to parallelize work across stations. Runtime remains limited by one model instance driving the loop.

(2) **Operational scale failure**. Models can show a stable mean per-station inference cost—for example ~ 0.13 s/station on CPU (Table 1)—yet total work grows roughly linearly with network size. In our mean timings, PhaseNet takes 32.58 s for 250 stations and 72.50 s for 580 stations (Table 1). Larger networks therefore exceed real-time deadlines even when the per-station rate is nearly flat.

(3) **Shared inference budget**. The 30-second real-time window must also cover data checks, phase association, and alert dispatch. At $N=228$, mean sequential `classify()` spans 1.47–28.63 s on CPU and 1.17–11.74 s on GPU across the four models (Table 1). On slower CPUs or under

competing load, picking alone can consume most or all of the 30 s budget, leaving little headroom for downstream stages.

(4) **Registry limitations.** Models not yet integrated into SeisBench, such as EQCCT or custom institutional pickers, lack a `classify()` path, forcing networks to find alternative multi-waveform processing solutions.

While parallelization is the intuitive remedy for these sequential processing constraints, treating each station as a separate parallel task forces the machine learning framework to re-initialize for every station. On our hardware, that cost dominates runtimes, leaving little margin for association, quality control, and dispatch.

To address these issues, we introduce **RAPID (Resource-Aware Parallel Inference Dispatcher)**, a generalized, resource-aware parallelization framework for seismic phase picking. RAPID is designed to complement SeisBench by using its native `from_pretrained()`, `annotate()`, and `classify()` interfaces while providing the necessary orchestration for real-time operations. We implement two parallelization strategies: **Model-Actor** (persistent instances) and **Ripper** (ephemeral tasks). We benchmarked these across five pickers and multiple hardware configurations. The Model-Actor method reduced total runtimes by roughly 50–87% compared with the best Ripper results at 228 stations for the same model–device pairs, demonstrating that persistent-actor orchestration yields faster end-to-end times and more comfortable real-time margins than tuned ephemeral-task approaches on real-world network workloads.

2 Methodology

The methodology is organized as follows: dataset and workload simulation (§2.1), model selection (§2.2), hardware environment and resource control (§2.3), orchestration strategies (§2.4), performance metrics (§2.5), and study protocol (§2.6).

2.1 Dataset and Workload Simulation

To evaluate our proposed parallelization strategies for real-time seismic processing, we used data from the Texas Seismological Network (TexNet; Savvaidis et al., 2019) to simulate realistic network-scale inputs. From TexNet’s 250 stations, we retrieved 228 unique three-component 60-second waveforms sampled at 100 Hz for a one-minute UTC window on 15 December 2024, distributed across West Texas. The 60-second window was chosen because it is the minimum input requirement for EQCCT (Saad et al., 2023) and matches TexNet’s standard operational interval. Although models such as PhaseNet can accept shorter windows, we used a uniform one-minute input across all models to ensure fair comparison of orchestration performance under identical workload conditions. We retrieved only 228 station records for that time window; the remaining 22 stations were either delayed, down, or in maintenance, further reflecting the challenges of real-time network processing.

Waveforms were pre-downloaded as miniSEED files to exclude network latency from the benchmark. During execution, waveforms are converted to NumPy arrays and stored in memory-resident Python dictionaries. This design isolates inference and orchestration costs from disk I/O. A 1–45 Hz bandpass filter was applied to all data during inference as a standard preprocessing step for high-frequency phase identification.

2.2 Model Selection

To demonstrate that the primary performance bottleneck is overhead rather than model architecture, we benchmarked the strategies across five models:

1. **PhaseNet** (U-Net; Zhu & Beroza, 2018),
2. **PhaseNetLight** (Optimized PhaseNet; Woollam et al., 2022)
3. **EQTransformer** (Attention-based; Mousavi et al., 2020)
4. **EQTransformer-NonConservative** (High-sensitivity configuration; Woollam et al., 2022)
5. **EQCCT** (Transformer-based; Saad et al., 2023).

These models utilize two machine learning frameworks: PyTorch (PhaseNet and EQTransformer variants) and TensorFlow (EQCCT). This selection of models covers a range of architectures: from lightweight U-Nets to computationally intensive transformers, allowing us to evaluate how framework choice and model complexity affect orchestration overhead.

2.3 Hardware Environment and Resource Control

Experiments ran on a workstation with dual AMD Ryzen Threadripper PRO 7985WX CPUs (128 cores total) and 512 GB of DDR5 RAM. GPU trials used two NVIDIA RTX 6000 Ada Generation GPUs, each with 49 GB of memory.

To simulate resource constraints seen in operational network deployments, we enforced strict hardware isolation. Using Linux’s `sched_setaffinity` utility, we restricted each trial to specific CPU cores and GPUs. We tested CPU allocations from 5 to 20 cores in steps of 3 (5, 8, 11, 14, 17, 20). For GPU trials, we applied the same CPU allocations while limiting inference to either one or two GPUs. These constraints reflect real-world deployment systems—for example, TexNet can dedicate up to 20 CPUs and up to two GPUs for deep learning operations. It is important to note that many analyst-grade laptops and workstations today are often shipped with on the order of 8–16 physical cores.

2.4 Orchestration Strategies

We evaluated three orchestration strategies that define how waveforms are processed across a station network.

2.4.1 Sequential Processing (Serial)

The serial workflow (Figure 1) loads a single model instance once and processes each waveform in order. SeisBench offers two serial modes: *offline batch mode* and *per-station streaming mode*. Offline batch mode is highly efficient but cannot handle real-time streaming, where waveforms arrive asynchronously. Per-station streaming processes each incoming waveform individually using the `classify()` method. While this avoids batching delays, it runs SeisBench’s full preprocessing pipeline for every station, making runtime scale linearly with the number of stations. Despite this limitation, per-station streaming is the most practical single-process solution for real-time applications.

2.4.2 Ephemeral Task-Based Parallelism (Ripper)

The **Ripper method** (Figure 2) treats each station as an independent *Ray task* (Moritz et al., 2018). Ray is a distributed computing framework that manages parallel execution across workers. Each Ray task loads the model, runs inference on that station’s three-component waveform, and then exits. Every station therefore incurs the full framework initialization cost (§3.2).

Memory is the binding constraint for parallelism: if too many workers start simultaneously, the machine hits out-of-memory (OOM) errors and the trial fails. Because PyTorch and TensorFlow allocate large buffers during model initialization (e.g., CUDA context setup, XLA-compiled graphs), we limit the number of concurrent Ripper tasks using calibrated per-task memory budgets (Table 2). For GPU trials, Ripper assigns a higher effective VRAM requirement per task than Model-Actor method to account for memory fragmentation and repeated CUDA context creation. We apply scaling factors from isolated testing ($1.7\times$ for PhaseNet and PhaseNetLight; $2.0\times$ for EQCCT and EQTransformer) plus extra headroom (Table 2).

Parallel Task Scheduling. To avoid exhausting resources, Ripper limits how many tasks can run simultaneously. Let N be the number of stations to process and R the maximum concurrent tasks allowed by memory constraints. The driver starts by launching up to $\min(R, N)$ tasks. As each task finishes, the driver collects its result and launches the next station task (if any remain). This keeps active tasks at or below R throughout execution (except at the end when fewer than R stations remain), ensuring memory usage stays within limits.

Waveform Referencing. The driver reads all miniSEED files once and combines them into three-component ObsPy `Stream` objects, along with processing parameters. These objects are stored in Ray’s shared object store using `put`. Each task receives only a reference to this shared data, retrieves it with `get`, extracts the traces for its assigned station (matching network and station codes), then runs preprocessing and inference. This prevents workers from redundantly reading the same files from disk. Ray’s reference counting keeps one copy in memory as long as any task needs it.

The **Concurrent Tasks** setting in each Ripper trial is the configured cap on concurrent station jobs (R above). It is always at or below the maximum safe R implied by Table 2 and the available RAM/VRAM. Within that band, we choose the value from the sweep in §2.6 that minimizes total trial time for that model and station count, rather than always using the largest feasible concurrency. That trade-off exists because more overlap shortens the sequential tail of station work but also raises peak memory use, Ray scheduling load, and contention when many frameworks start or run together.

2.4.3 Persistent Inference Actors (Model-Actor)

The **Model-Actor method** (Figure 3) uses Ray *actors*—stateful, long-lived workers that persist across calls. Actors are like persistent workers that stay on standby rather than being allocated and deallocated per station task. Each actor loads its PyTorch or TensorFlow weights once and keeps them resident in RAM or VRAM for the duration of the processing window.

The system initializes the requested number of actors, capping this count based on available host memory. After initialization, the driver waits for each actor to respond to a `ready()` call, confirming the model is fully loaded before any station processing begins. This concentrates the model-loading overhead at startup rather than during inference.

Parallel Task Scheduling. Station processing uses lightweight Ray tasks: each task extracts one station’s waveform data, prepares the input, and sends it to an actor for inference. The driver assigns stations to M actors in round-robin order (station i goes to actor $i \bmod M$), distributing stations evenly across actors (station 1 to actor 1, station 2 to actor 2,..., station $M+1$ back to actor 1).

Unlike Ripper, actors never reload the model—they keep it in memory until all stations are processed, then shut down. As with Ripper, we limit simultaneous actors based on hardware constraints (Table 2). The driver submits tasks until hitting the limit of available actors, then waits for completions before submitting more, preventing system overload.

Waveform Referencing. Model-Actor uses the same approach as Ripper: the driver reads waveforms once, stores them in Ray’s object store with `put`, and passes references to tasks. Each task retrieves and subsets the data for its station before the actor runs inference.

The **Actors** count is bounded above by host RAM or VRAM divided by the per-actor budgets in Table 2. The specific **Actors** values in our experiments are taken from the sweep in §2.6: we increase the pool until total trial time stops improving or the run becomes unstable. Additional actors add one-time setup and scheduling overhead, and multiple actors on one GPU or CPU set can contend for the same device. The minima in Table 5 therefore reflect tuned parallelism, not a fixed "use as many actors as memory allows" policy.

2.5 Performance Metrics

We measured two primary timing metrics: **Total Trial Time** and **Total Run Time for Picker**. Total Trial Time captures the complete wall-clock duration from reading waveforms through saving results, including all setup costs, model loading, and orchestration overhead. Total Run Time for Picker measures only the time spent on inference and preprocessing, isolating the core computational cost of the picking algorithm.

We monitored memory usage continuously throughout each trial using `psutil` (a Python library for measuring RAM) and `pynvml` (NVIDIA’s library for measuring GPU memory). To determine safe concurrency limits, we first measured how much memory each method requires per worker in isolated tests, accounting for framework initialization, model weights, and inference buffers. We then added safety buffers (1024 MB for GPU memory, 1536 MB for RAM) to handle Ray’s overhead and temporary memory spikes. Finally, to prevent out-of-memory errors, we calculated the maximum number of concurrent workers by dividing available memory by the per-worker budget and applying a 95% safety cap (Table 2).

2.6 Study Protocol

Each configuration was tested across a range of station counts (5, 10, 15, $\dots$, 225, 228). Concurrency levels were tested in 20% increments up to the memory-limited maximum for each strategy. Trials that failed due to out-of-memory errors or system instability were re-run until successful completion. No other workloads ran on the workstation during our trials, ensuring maximum available compute.

3 Results

3.1 Single-Process Inference Baselines

To establish a reference for the parallel strategies, we measured model load time, merged-stream `annotate()` (batch picking), and sequential `classify()` (streaming picking) for the four SeisBench models on CPU and on one GPU at representative network sizes ($N \in \{228, 250, 580\}$; Table 1). EQCCT is excluded from this baseline because it uses a custom TensorFlow loader without SeisBench’s standard stream pipeline, limiting comparable end-to-end preprocessing and inference conditions across models. All values in Table 1 are means over five repeated runs.

Warm-cache model load times were consistent across models (1.18–1.20 s, Table 1). Offline batch picking via merged-stream `annotate()` stayed near 0.24–0.52 s at 228 stations across CPU and GPU. Streaming picking via sequential `classify()` grows roughly linearly with N . On CPU, PhaseNet approaches the 30 s target at 228 stations (28.63 s) and exceeds it at 250 stations (32.58 s). On one GPU, all models meet the 30 s target at 228 and 250 stations, but EQTransformer slightly exceeds it at 580 stations (30.90 s). This linear scaling leaves little operational headroom for association and alerting at larger N .

3.2 Ripper: Ephemeral Task-Based Parallelism

Across CPU Ripper runs (Table 3), total runtimes for 228 stations range from 34.3–76.1 s, while GPU Ripper configurations range from 52.5–127.4 s. The slowest GPU Ripper results occur for EQCCT, driven by repeated re-initialization of TensorFlow XLA-compiled graphs in each ephemeral task. In contrast, the PyTorch-based SeisBench models achieve more consistent CPU performance, typically 34–38 s. On GPU, these same models range from 52.5–67.6 s, depending on model and GPU count, with some configurations performing better on two GPUs than on one (Table 3).

For models with both CPU and GPU Ripper results, GPU runtimes are generally slower due to repeated framework initialization and CUDA context setup per task. For example, PhaseNet’s best GPU Ripper configuration (two GPUs, 56.3 s) is approximately 64% slower than its best CPU Ripper result (34.3 s) for 228 stations.

3.3 Model-Actor: Persistent Inference Actors

The Model-Actor method yielded the lowest runtimes among the evaluated parallel strategies (Table 5). At each model’s optimal configuration, actor counts reached up to 46 on CPU-only systems for several SeisBench models, 22–44 in GPU configurations depending on the model and number of GPUs, and 12–24 for EQCCT on GPU (with 46 on CPU).

End-to-end runtimes range from 10.97 s (PhaseNet, one GPU) to 25.01 s (EQCCT on CPU). For EQCCT in particular, the Model-Actor method highlights the benefit of one-time initialization: the fastest one-GPU configuration (12.47 s) is approximately 87% faster than the fastest GPU Ripper result (96.10 s, Table 3), which itself outperforms the one-GPU Ripper configuration (127.35 s).

The Model-Actor approach incurs a one-time setup cost during actor initialization. In Table 5, setup ranges from 4.6–12.9 s, accounting for roughly 37–73% of total runtime depending on configuration. This overhead is amortized across all stations in the batch and becomes negligible on a per-station basis under continuous operation with long-lived actors.

The **Actors** column lists the persistent-worker pool size for each row, selected under the same sweep as in §2.4.3 and §2.6 within the Table 2 memory bounds. The counts reflect a trade-off between paying actor-setup cost once and supplying enough parallel inference paths to finish the window quickly without oversubscribing the GPU or CPU cores; Table 5 shows the measured RAM/VRAM footprint at those choices.

3.4 Memory Utilization

Memory tracking confirmed that the actor budgeting system prevented OOM errors at the Model-Actor configurations summarized in Table 6. On CPU, the SeisBench optima typically use 46 actors; measured process-tree RAM stays far below the pre-allocated request because the budget includes large safety margins. EQTransformer-NC shows the largest CPU tree footprint in that table. EQCCT CPU follows the same pattern at 46 actors and 14 Ray CPUs. Without conservative

budgeting, early sweeps hit OOM during concurrent actor startup; the Ripper VRAM scaling factors in §2.4.2 were chosen for the same reason.

On GPU, the one-GPU configurations in Table 5 typically use around 22 actors for SeisBench models and 12 actors for EQCCT. Two-GPU configurations can scale to 44 or 24 actors, respectively, when those settings yield the best performance. These values remain below the hardware-specific limits. Pushing to the maximum number of actors did not minimize total runtime. In fact, increasing actor count beyond the optimal point tends to degrade performance by increasing both picking and overall execution time. This is likely due to increased Ray scheduling and inter-process communication overhead, resource contention when many actors share the same GPU or CPU cores, and queuing effects once device throughput is saturated. As a result, additional actors primarily increase coordination costs rather than improving inference capacity.

For EQCCT, GPU memory usage would occasionally exceed the nominal per-actor VRAM budget due to additional allocations from TensorFlow and XLA. However, all observed values remained within the physical 49 GB per-GPU limit.

4 Discussion

4.1 The Case for Persistent-Actor Orchestration in Real-Time Deep Learning Phase Picking

In our study, the primary runtime bottleneck in stream-based seismic phase picking is per-task initialization cost rather than inference complexity. The Ripper method makes this explicit: warm-cache model load time is about 1.16–1.20 s (Table 1), so repeating that cost across hundreds of stations can dominate wall time and push GPU Ripper and EQCCT on Ripper CPU above the 30 s real-time threshold.

The Model-Actor strategy removes this bottleneck by loading model instances once and reusing them across all incoming waveforms, reducing the marginal cost per station to forward-pass inference time plus inter-process communication (IPC) latency.

Among the approaches that remain practically viable for real-time regional picking at 228 stations, **Model-Actor is the clear winner**: it is the only strategy that combines sub-30 s trial times for every model we tested (Table 5) with a deployment shape that matches streaming operation. Serial `classify()` is too slow at scale for large networks despite nearly stable per-station costs (Table 1), and merged-batch `annotate()` does not match asynchronous arrivals. Ripper can parallelize work, but repeated model startup inside many short-lived Ray tasks inflates end-to-end time and leaves our Ripper runs above the 30 s target at 228 stations (Table 3), whereas Model-Actor keeps totals low enough to leave room for association and dispatch.

At 228 stations, Model-Actor achieves end-to-end runtimes below the 30-second real-time target for all configurations in Table 4, while GPU Ripper and EQCCT on CPU remain above this threshold. Figures 5–6 show runtime scaling across station counts (5–228) for both methods, with serial baselines anchored to the dense CPU serial benchmark (Table 1 lists $N \in \{228, 250, 580\}$ only). Persistent-actor curves track closer to streaming-like scaling, with a positive offset from one-time actor setup and IPC overhead. For compute-intensive models, parallel execution reduces the marginal cost per station enough to offset this initialization overhead as network size increases.

Although neither parallelization approach reaches the theoretical batch inference limit (Amdahl’s ideal curve; Amdahl 1967), this gap is expected. Batch `annotate()` bypasses the per-station processing pipeline entirely, while both parallel strategies incur IPC overhead. GPU-based Ripper configurations can approach this limit more closely than CPU-based ones due to lower per-task compute time, but still remain well above the ideal curve.

Runtimes for both methods improve as more CPU cores are allocated to Ray, increasing the number of concurrent workers and reducing the sequential tail of each run. This is evident in Figure 4, where Ripper total-time curves that rise steeply with station count at low CPU counts flatten as more cores are added. Model-Actor curves already follow a shallower trajectory, reflecting more efficient utilization of parallel inference. This trend is reinforced in Figures 5–6, where Model-Actor reaches lower runtimes more quickly and approaches the theoretical performance limit more closely than Ripper across comparable configurations.

For Model-Actor in particular, tuning the actor pool under Table 2 and the sweep in §2.6—rather than fixing an arbitrary worker count—is what keeps end-to-end picking within the 30 s real-time window at 228 stations while limiting one-time setup and on-device contention. Ripper benefits from the analogous tuning of concurrent task caps, though its repeated cold starts still leave our Ripper trials above that threshold at 228 stations (Table 3).

5 Conclusion

RAPID is a resource-aware parallelization framework that enables real-time, network-scale seismic phase picking. By combining persistent model actors with hardware-constrained memory budgeting, RAPID processes 228 stations in **10.97–25.01 s** under the Model-Actor configurations—representing approximately **50–87%** lower total runtime than the corresponding best Ripper results. It meets the 30-second real-time target for every Model-Actor configuration in Table 5, including compute-intensive models such as EQCCT. These outcomes were obtained with at most 20 allocated CPUs, so networks and institutions without access to large clusters can still deploy RAPID on modest, memory-adequate hardware. Networks with larger computing resources can allocate additional resources to further reduce runtimes.

These results confirm that orchestration overhead—specifically repeated model loading and teardown—dominates runtime in the Ripper approach at network scale, keeping it above the 30-second threshold. In contrast, Model-Actor configurations consistently achieve sub-30 s runtimes. RAPID is model-agnostic, supporting both PyTorch and TensorFlow through a unified interface, and is designed to complement existing tools such as SeisBench by providing the orchestration layer required for real-time, streaming waveform processing.

6 Limitations and Future Work

6.1 Current Limitations

This study evaluated performance on a single workstation. While internal testing at TexNet suggests these gains are consistent across similar hardware, formal multi-node evaluations for larger networks (e.g., NCSN or SCSN) have not yet been conducted. Additionally, waveforms were pre-loaded into RAM to isolate orchestration costs; live deployments would incur extra overhead for waveform decoding and network retrieval. The 20% step size in how we varied concurrency also means the reported optimal actor counts are approximations that could be refined with more granular testing. Finally, EQCCT is currently accessed through a custom TensorFlow loader; porting it to PyTorch for native SeisBench integration would likely reduce framework-specific initialization costs.

6.2 Operational Deployment via SeisComP Integration

The nearest-term deployment target is SeisComP, the open-source platform used by TexNet and other national networks. Chen, Savvaidis, et al. (2024) demonstrate that EQCCT already integrates

with SeisComP for event association and catalog generation. RAPID is designed to integrate directly into this ecosystem: it orchestrates Ray workers, batches data into SeisBench or TensorFlow pipelines, and ensures feasible wall-clock runtimes when all stations in a processing window must be picked prior to association.

TexNet is extending this workflow so that live miniSEED data streams can feed into Model-Actor-based picking within the same processing cycle as the broader catalog pipeline, eliminating the need for ad hoc resource management on a per-event basis.

6.3 EQCCT Integration into SeisBench

EQCCT is currently implemented via TensorFlow, while SeisBench requires PyTorch for integration with its training infrastructure and community registry. Porting EQCCT to PyTorch is underway at TexNet. Once integrated, EQCCT will benefit from SeisBench’s standardized preprocessing and weight management, while gaining immediate compatibility with RAPID’s PyTorch-based memory budgeting. This will remove the TensorFlow-specific XLA compilation overhead and potentially reduce actor setup time relative to model complexity.

6.4 RAPID as a SeisBench-Compatible Tool

A long-term goal is contributing RAPID to SeisBench as a standardized orchestration module. This would allow any SeisBench-compatible model to run in persistent-actor streaming mode with built-in hardware-aware memory budgeting and multi-GPU scheduling. This would lower the engineering barrier for networks looking to deploy DL picking in production settings.

6.5 Distributed and Dynamic Scaling

Ray natively supports multi-node clusters, which would allow RAPID to scale to networks exceeding single-machine capacity. Future work will investigate how memory budgeting behaves across heterogeneous nodes and how scheduling overhead impacts performance at very large scales. Additionally, a dynamic scaling mode that adjusts the actor pool based on real-time resource usage would remove the need for offline calibration and improve reliability on shared infrastructure. Finally, alternative parallelization toolkits must be explored to identify further techniques that will lower processing runtimes.

7 Data and Resources

Seismic wave data and computational hardware were provided by the Texas Seismological Network and Seismology Research Team (TexNet). All seismic data were downloaded from TexNet’s FDSN network and are publicly available at <http://rtserve.beg.utexas.edu/>. EQCCT is an open-source machine learning model (Saad et al. (2023)) and is available on GitHub at <https://github.com/ut-beg-texnet/eqcct/tree/main>. EQCCTone and RAPID are also open-source and can be accessed at <https://github.com/ut-beg-texnet/eqcct/tree/main/eqcctone> and <https://github.com/ut-beg-texnet/eqcct/tree/main/eqcctpro>, respectively. The `eqcctpro` tree includes **evaluation and benchmark scripts** (under `eqcctpro/experiments/workbench/scaling/`) for serial SeisBench spot-checks and orchestration trials, and **figure-generation utilities** (under `eqcctpro/scripts/visualization/`) that rebuild the runtime and scaling plots from recorded JSON and trial outputs. With local waveform data and dependencies installed, users can run these

scripts on their own machines to reproduce wall-clock timings and figures, subject to GPU drivers, memory, and dataset paths.

8 Acknowledgements

The authors would like to thank the Texas Seismological Network and Seismology Research (TexNet) group and the State of Texas that provided financial support for this publication under the University of Texas at Austin award #201503664. Special thanks to Elena Kalogirou and Peter Sarkis for revising this paper.

9 References

- Sheen, D.-H., Y.-J. Seong, S. Jung, and J.-h. Park (2023). Real-time application of deep learning seismic phase picker to earthquake monitoring. Poster S31G-0424, *AGU Fall Meeting*, San Francisco, CA, 11–15 December, session *Machine Learning–Driven Analysis of Geophysical Signals III*.
- Ali, M. (2023, Jul). Distributed processing using ray framework in python.
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, AFIPS '67 (Spring)*, New York, NY, USA, pp. 483–485. Association for Computing Machinery.
- Bates, D. and D. Watts (2008, 05). *Nonlinear Regression Analysis and Its Applications*, pp. 32 – 66.
- Chen, Y., O. M. Saad, A. Savvaidis, F. Zhang, Y. Chen, D. Huang, H. Li, and F. Aziz Zanjani (2024). Deep learning for p-wave first-motion polarity determination and its application in focal mechanism inversion. *IEEE Transactions on Geoscience and Remote Sensing* 62, 1–11.
- Chen, Y., Savvaidis, A., Siervo, D., Huang, D., & Saad, O. M. (2024). Near real-time earthquake monitoring in Texas using the highly precise deep learning phase picker. *Earth and Space Science*, 11, e2024EA003890. <https://doi.org/10.1029/2024EA003890>
- Lim, C. S. Y., S. Lapins, M. Segou, and M. J. Werner (2025). Deep learning phase pickers: how well can existing models detect hydraulic-fracturing induced microseismicity from a borehole array? *Geophysical Journal International* 240(1), 535–549, <https://doi.org/10.1093/gji/ggae386>
- Feng, T., S. Mohanna, and L. Meng (2022). Edgephase: A deep learning model for multi-station seismic phase picking. *Geochemistry, Geophysics, Geosystems* 23(11), e2022GC010453. e2022GC010453 2022GC010453.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29(5), 1189 – 1232.
- Gustafson, J. L. (1988, May). Reevaluating amdahl’s law. *Commun. ACM* 31(5), 532–533.
- Herlihy, M. and N. Shavit (2012). *The Art of Multiprocessor Programming*, Revised Reprint. Morgan Kaufmann.
- Johnston, J. and J. DiNardo (1997). *Econometric Methods* (4th ed.). McGraw-Hill.
- McCool, M., J. Reinders, and A. Robison (2012). *Structured Parallel Programming: Patterns for Efficient Computation*. ITPro collection. Elsevier Science.
- Moritz, P., R. Nishihara, S. Wang, A. Tumanov, R. Liaw, X. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica (2018). Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, Carlsbad, CA, pp. 561–577. USENIX Association.
- Mousavi, S., W. Ellsworth, Z. Weiqiang, L. Chuang, and G. Beroza (2020, 08). Earthquake transformer—an attentive deep-learning model for simultaneous earthquake detection and phase

picking. *Nature Communications* 11, 3952.

Mousavi, S. M. and G. C. Beroza (2022). Deep-learning seismology. *Science* 377(6607), eabm4470.

Münchmeyer, J., J. Woollam, A. Rietbrock, F. Tilmann, D. Lange, T. Bornstein, T. Diehl, C. Giunchi, F. Haslinger, D. Jozinović, A. Michelini, J. Saul, and H. Soto (2022, 01). Which picker fits my data? a quantitative evaluation of deep learning based seismic pickers. *Journal of Geophysical Research: Solid Earth* 127.

Ray.io (n.d.). What is ray core? <https://docs.ray.io/en/latest/ray-core/walkthrough.html>. Accessed: 2023-11-13.

Saad, O. M. and Y. Chen (2022). Capsphase: Capsule neural network for seismic phase classification and picking. *IEEE Transactions on Geoscience and Remote Sensing* 60, 1–11.

Saad, O. M., Y. Chen, A. Savvaidis, W. Chen, F. Zhang, and Y. Chen (2022). Unsupervised deep learning for single-channel earthquake data denoising and its applications in event detection and fully automatic location. *IEEE Transactions on Geoscience and Remote Sensing* 60, 1–10.

Saad, O. M., Y. Chen, A. Savvaidis, S. Fomel, and Y. Chen (2022). Real-time earthquake detection and magnitude estimation using vision transformer. *Journal of Geophysical Research: Solid Earth* 127(5), e2021JB023657. e2021JB023657 2021JB023657.

Saad, O. M., Y. Chen, D. Siervo, F. Zhang, A. Savvaidis, G.-c. D. Huang, N. Igonin, S. Fomel, and Y. Chen (2023). Eqcct: A production-ready earthquake detection and phase-picking method using the compact convolutional transformer. *IEEE Transactions on Geoscience and Remote Sensing* 61, 1–15.

Saad, O. M., G. Huang, Y. Chen, A. Savvaidis, S. Fomel, N. Pham, and Y. Chen (2021). Scalodeep: A highly generalized deep learning framework for real-time earthquake detection. *Journal of Geophysical Research: Solid Earth* 126(4), e2020JB021473. e2020JB021473 2020JB021473.

Savvaidis, A., B. Young, G. D. Huang, and A. Lomax (2019, 06). Texnet: A statewide seismological network in texas. *Seismological Research Letters* 90(4), 1702–1715.

Tan, Y. J., F. Waldhauser, W. Ellsworth, M. Zhang, Z. Weiqiang, M. Michele, L. Chiaraluce, G. Beroza, and M. Segou (2021, 04). Machine-learning-based high-resolution earthquake catalog reveals how complex fault structures were activated during the 2016–2017 central italy sequence. *The Seismic Record* 1, 11–19.

Wang, T., D. Trugman, and Y. Lin (2021). Seismogen: Seismic waveform synthesis using gan with application to seismic data augmentation. *Journal of Geophysical Research: Solid Earth* 126(4), e2020JB020077. e2020JB020077 2020JB020077.

Woollam, J., J. Münchmeyer, F. Tilmann, A. Rietbrock, D. Lange, T. Bornstein, T. Diehl, C. Giunchi, F. Haslinger, and D. Jozinović (2022). SeisBench—A Toolbox for Machine Learning in Seismology. *Seismological Research Letters* 93(3), 1695–1709.

Xiao, Z., J. Wang, C. Liu, J. Li, L. Zhao, and Z. Yao (2021). Siamese earthquake transformer: A pair-input deep-learning model for earthquake detection and phase picking on a seismic arrival-time picking method. *Journal of Geophysical Research: Solid Earth* 126(5), e2020JB021444. e2020JB021444 2020JB021444.

Yeck, W. L., J. M. Patton, Z. E. Ross, G. P. Hayes, M. R. Guy, N. B. Ambruz, D. R. Shelly, H. M. Benz, and P. S. Earle (2020). Leveraging Deep Learning in Global 24/7 Real-Time Earthquake Monitoring at the National Earthquake Information Center, *Seismol. Res. Lett.* 92, 469–480, doi: 10.1785/0220200178.

Yu, Z., Y. Jiang, X. Jing, and H. Zheng (2024). Study on geomagnetic observations associated with three major earthquakes in southwest china through a novel deep learning framework. *IEEE Transactions on Geoscience and Remote Sensing* 62, 1–18.

Zhu, W. and G. C. Beroza (2018, 10). Phasenet: a deep-neural-network-based seismic arrival-time picking method. *Geophysical Journal International* 216(1), 261–273.

Material

The following pages contain **Tables 1–5** and **Figures 1–8**: workflow diagrams (1–3), runtime surfaces and scaling comparisons (4–6), and 228-station summaries plus memory bars (7–8).

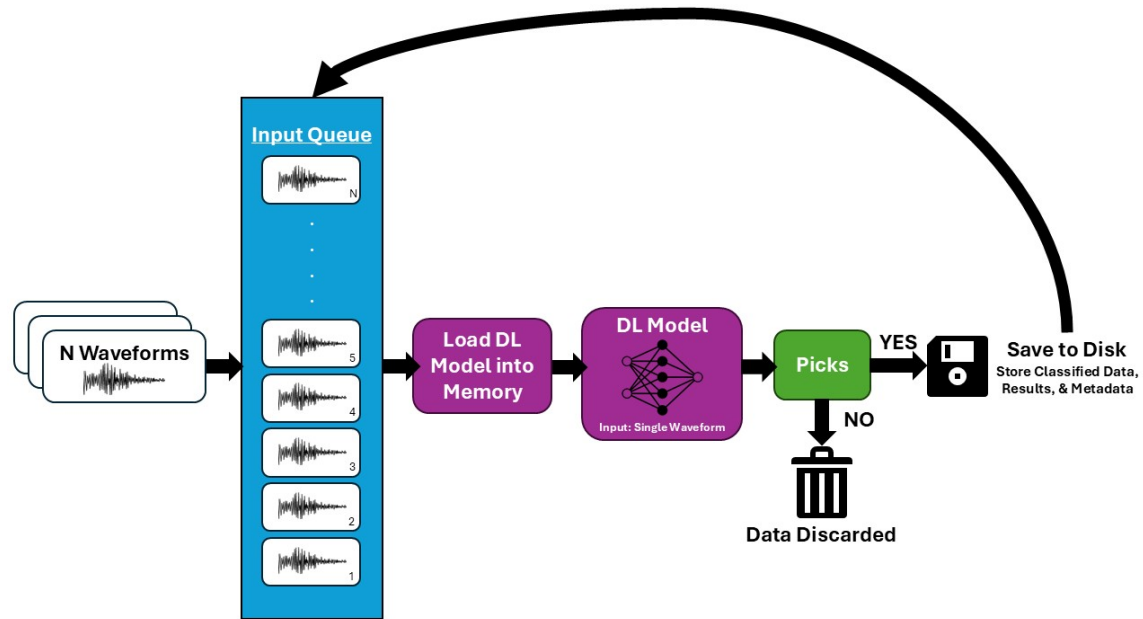


Figure 1. Serial baseline workflow. Waveforms are processed sequentially by a single DL model instance. Each waveform must wait for the previous one to complete, creating a linear bottleneck that scales poorly with network size.

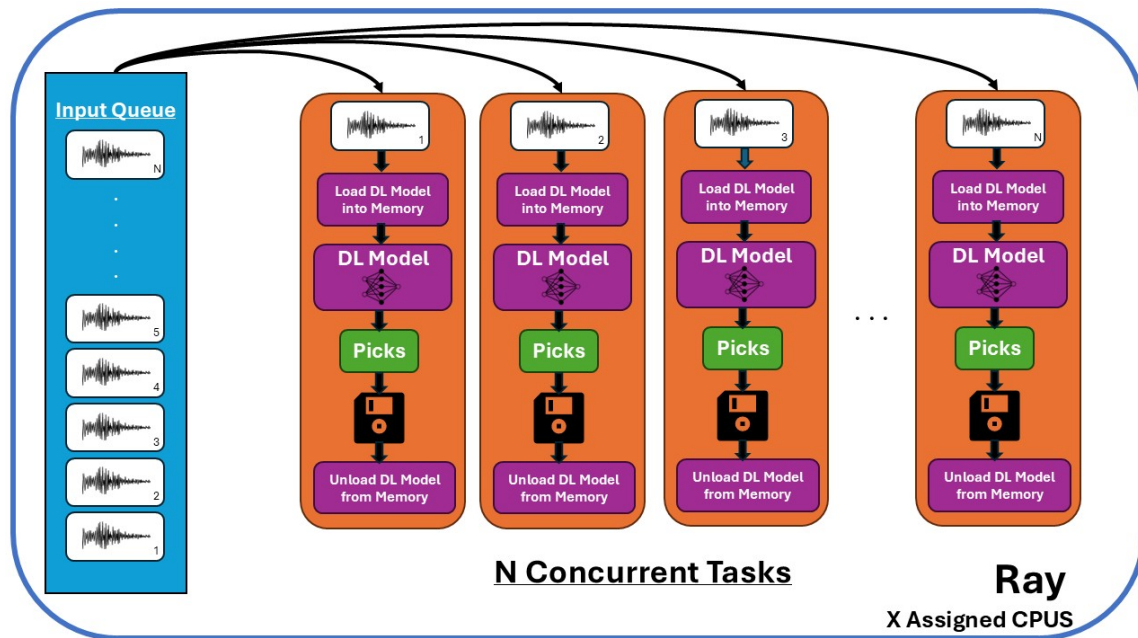


Figure 2. Ripper workflow. Each task independently loads the model, performs inference, and unloads. Multiple tasks run in parallel but incur repeated framework initialization overhead per station.

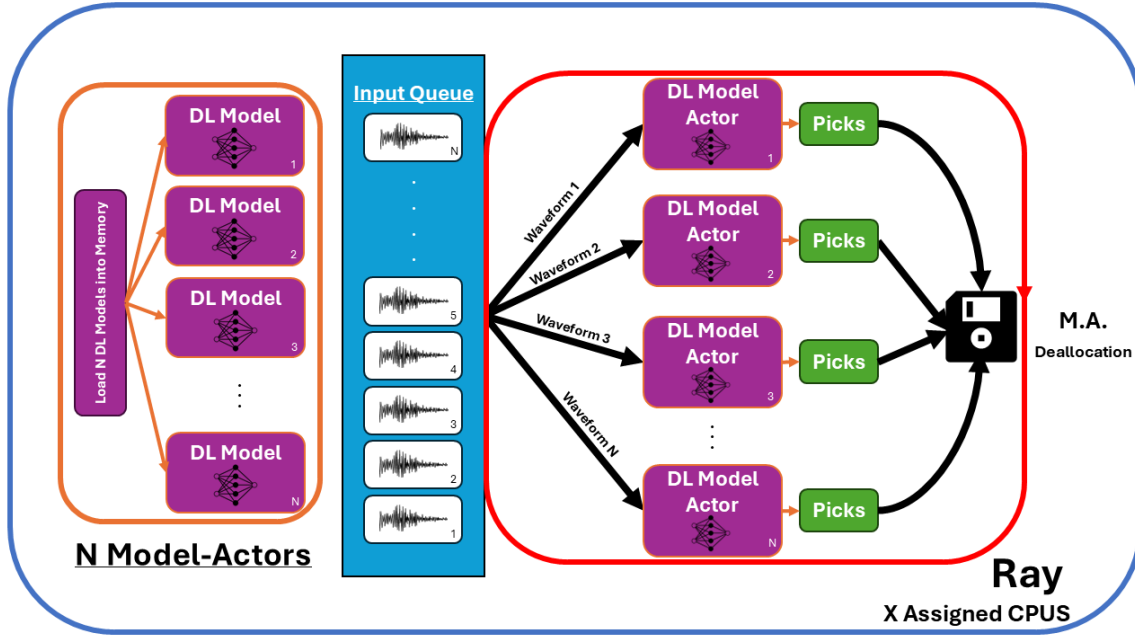


Figure 3. Model-Actor workflow. Persistent inference actors maintain loaded models in memory (or GPU VRAM). Waveforms are dispatched to actors as a stream, eliminating load/unload overhead and maximizing throughput.

Total Trial Runtime vs Station Count by CPU Allocation

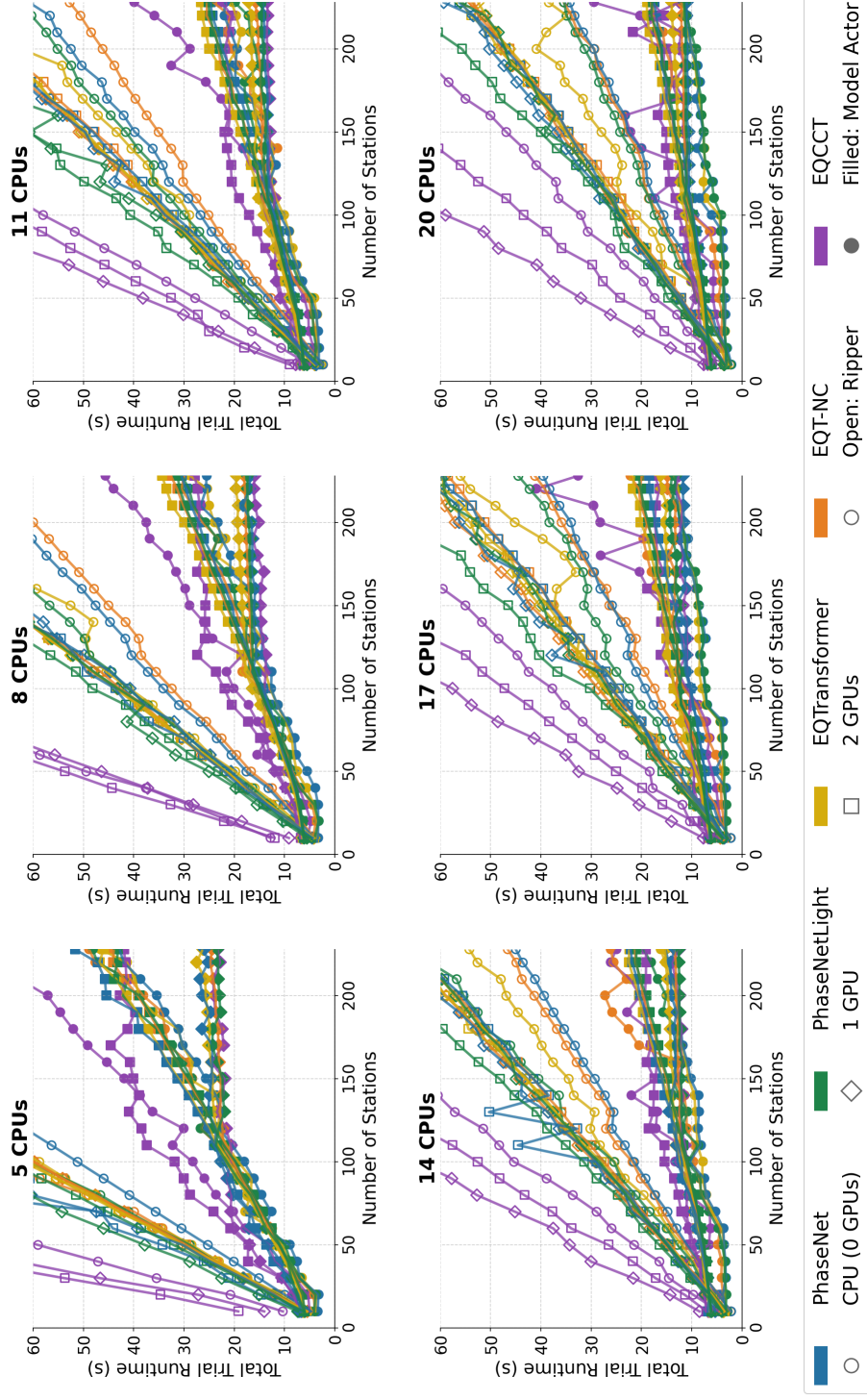


Figure 4. Comparison of total trial runtime across all models and parallelization methods. Data points are plotted every 10 stations from 10 to 228. Fig. 7 and Table 3 summarize best 228-station totals.

Serial vs Fastest Ripper vs Amdahl Ideal

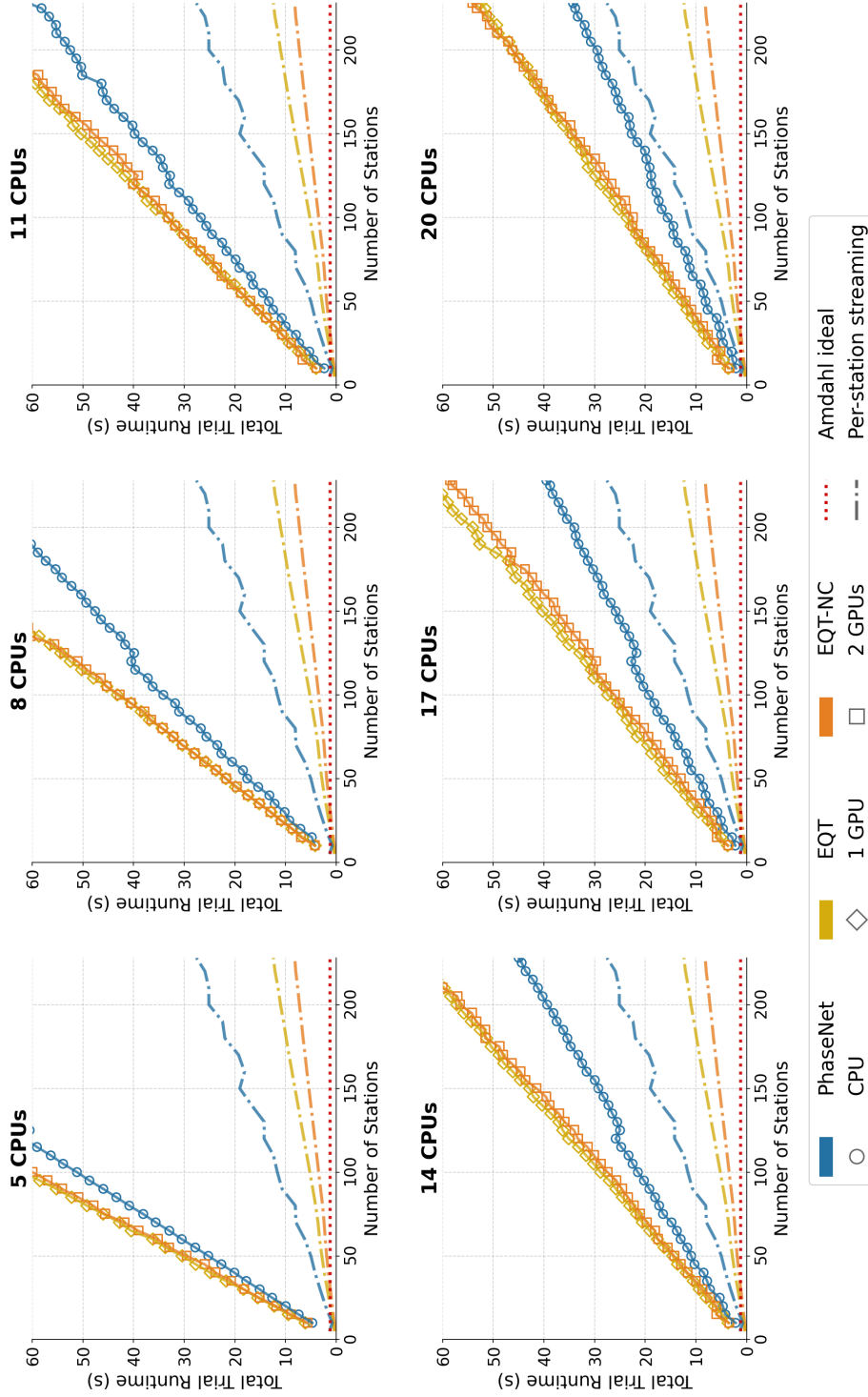


Figure 5. Serial baseline versus fastest Ripper configurations, with the batch-based Amdahl reference shown for each CPU allocation. For each Ripper hardware class, a single model is selected from the tested pool based on the lowest *mean* total runtime across successful trials; this model defines the Ripper curve in each panel. Serial curves use the dense CPU serial benchmark (interpolated between anchors; Table 1 lists $N \in \{228, 250, 580\}$ only). The figure emphasizes scaling with network size rather than the individual minima reported in Table 3.

Serial vs Fastest Model Actor vs Amdahl Ideal

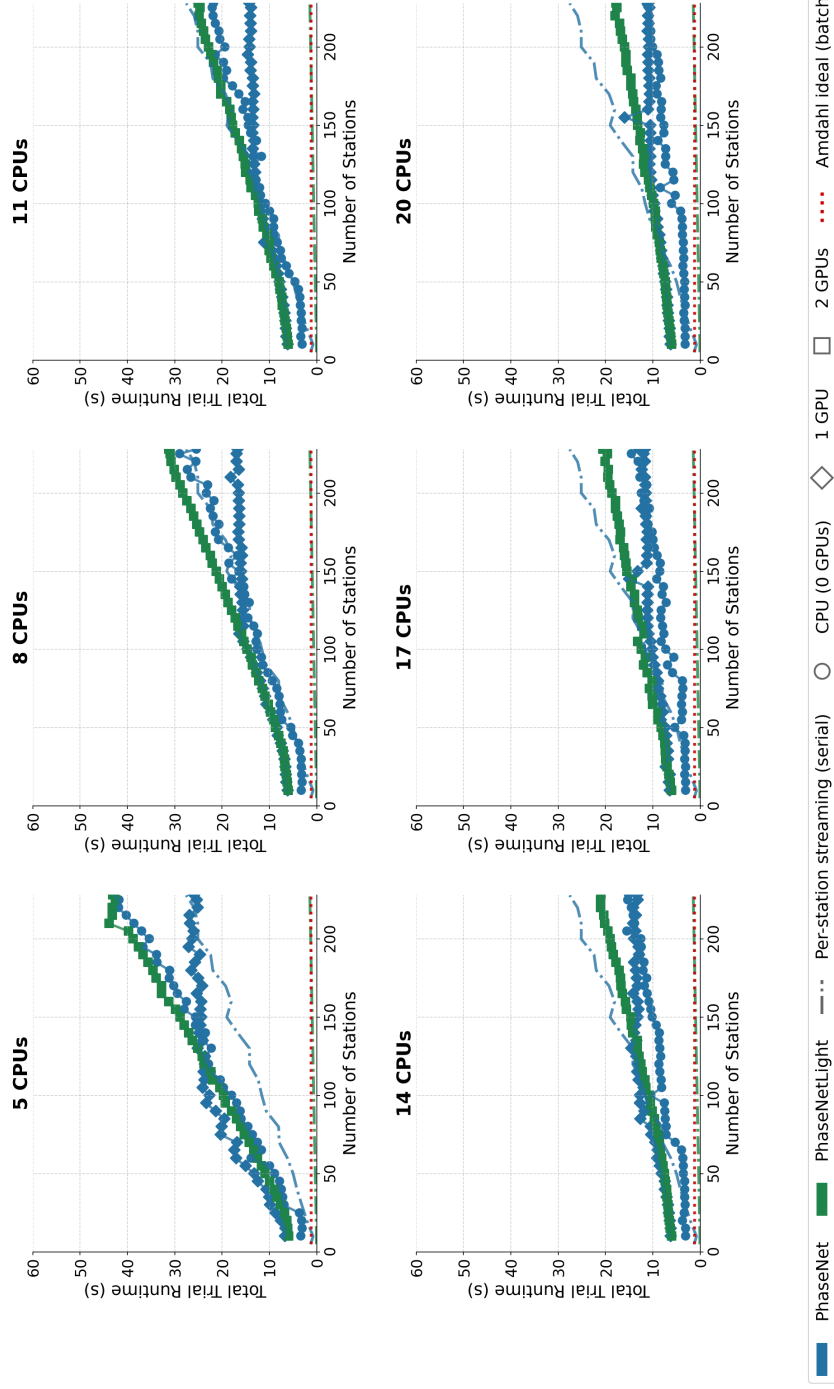


Figure 6. Serial baselines versus fastest Model-Actor configurations, with the Amdahl ideal limit shown for each CPU allocation. As in Figure 5, a single model is selected based on the lowest *mean* total runtime across successful trials; this model defines the Model-Actor curve in each panel. Serial curves use the dense CPU serial benchmark (Table 1 lists $N \in \{228, 250, 580\}$ only). The red dotted line denotes the Amdahl ideal, representing the theoretical minimum runtime under perfectly scalable batch processing.

Ripper vs. Model-Actor: minimum total runtime at 228 stations
(Ray CPUs 5-20; Ripper GPU bar = faster of best 1-GPU vs 2-GPU Ripper; MA: min total in grid)

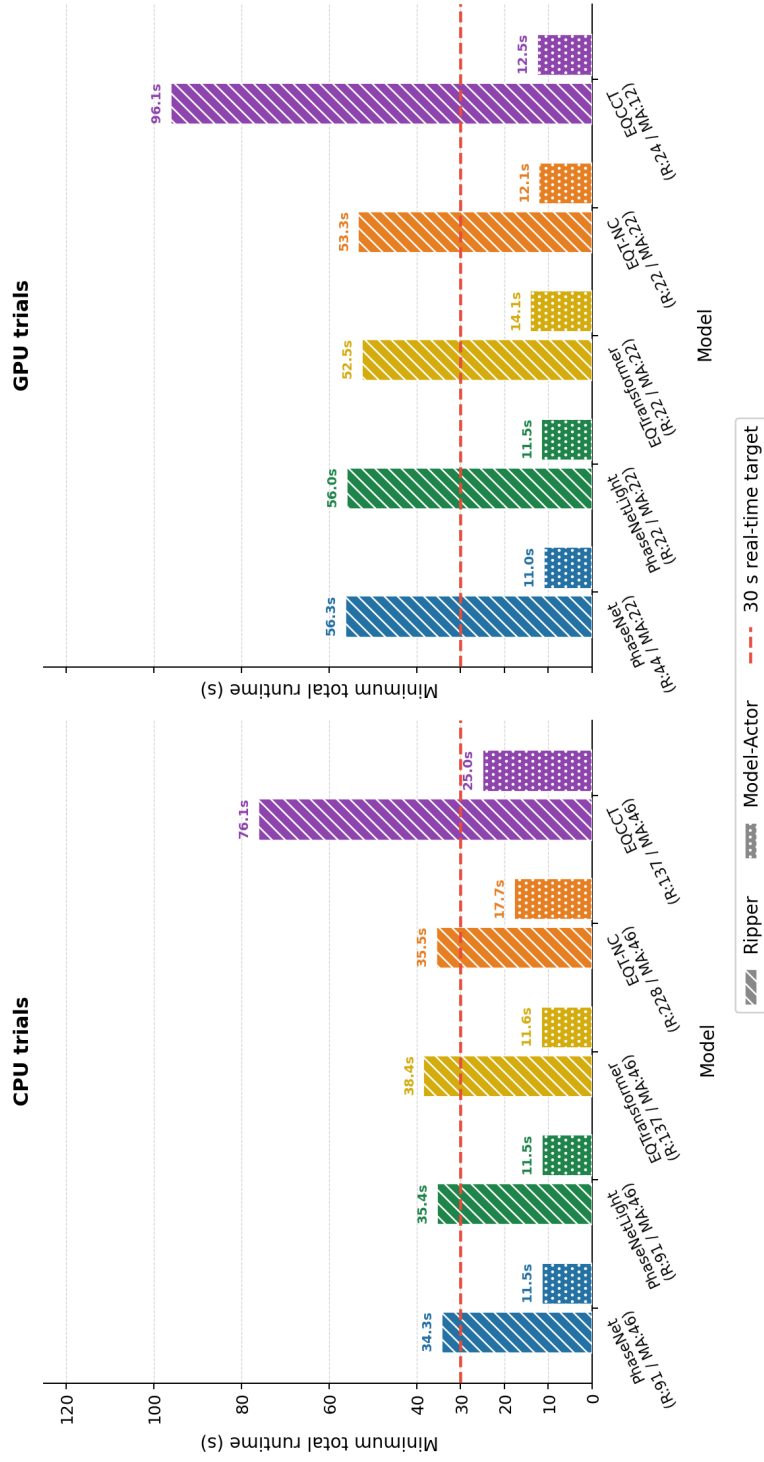


Figure 7. Minimum total runtime at 228 stations for Ripper versus Model-Actor (left: CPU, right: GPU). Bars are best successful end-to-end times at 228 stations per strategy. Dashed red line: 30 s target.

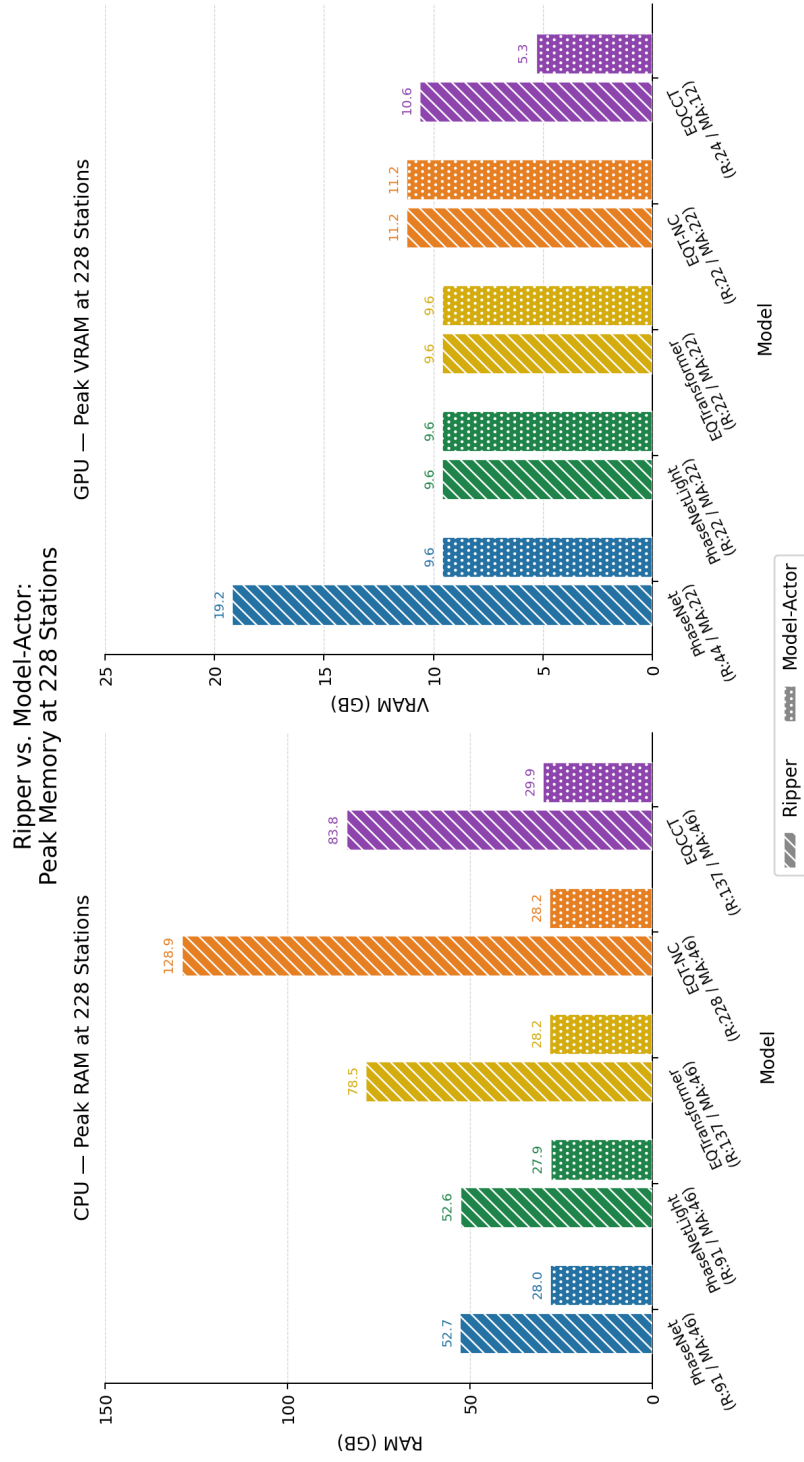


Figure 8. Peak process-tree RAM (CPU) and VRAM (GPU) from the load-only benchmark: all N workers held resident while sampling; bars show the maximum over 25 s samples (1.25 s interval). N from best 228-station trial rows; R = Ripper tasks, MA = actors.

Table 1. Single-process SeisBench baselines on CPU and on one GPU at $N \in \{228, 250, 580\}$ (mean over five runs per quantity). **Ld** = model load time (fresh load cycle); **Ann** = merged-stream `annotate()` wall time for all N stations; **Cls tot** = total wall time for one full sequential `classify()` pass over all N stations. Per-station `classify()` time is **Cls tot**/ N . All times in seconds.

Model	Ld	N=228		N=250		N=580	
		Ann	Cls tot	Ann	Cls tot	Ann	Cls tot
CPU							
PhaseNet	1.187	0.279	28.628	0.293	32.580	0.694	72.504
PhaseNetLight	1.202	0.264	1.473	0.298	1.618	0.634	3.759
EQTransformer	1.182	0.524	12.422	0.530	13.630	1.490	31.923
EQT-NC	1.186	0.475	8.208	0.532	8.995	1.334	21.086
GPU (one GPU)							
PhaseNet	1.194	0.241	2.224	0.248	2.417	0.599	5.559
PhaseNetLight	1.176	0.235	1.171	0.257	1.291	0.583	2.959
EQTransformer	1.175	0.382	11.740	0.393	12.989	0.921	30.902
EQT-NC	1.181	0.337	6.698	0.344	7.355	0.883	17.166

Table 2. Per-instance memory budgets (MB) used by RAPID to cap concurrency and avoid out-of-memory errors.

Model	Framework	Host RAM – CPU (MB)		Host RAM – GPU (MB)		GPU VRAM (MB)		
		Base	Per Ripper/Actor	Base	Per Ripper/Actor	Base	Per Ripper	Per Actor
PhaseNet	PyTorch	502	2,038	870	2,406	500	850	1,524
PhaseNetLight	PyTorch	502	2,038	861	2,397	500	850	1,524
EQTransformer	PyTorch	521	2,057	1,001	2,537	528	1,056	1,552
EQT-NC	PyTorch	524	2,060	1,017	2,553	530	1,060	1,554
EQCCT	TensorFlow	728	2,264	2,311	3,847	1,732	3,464	2,756

Table 3. Best successful Ripper total time at 228 stations, sorted by fastest time. CPU rows are minima over Ray CPUs tested; GPU rows report the best one-GPU and two-GPU Ripper totals separately.

Model	Device	CPUs	GPUs	Conc. Tasks	Ripper Picking/Total (s)
PhaseNet	CPU	20	0	91	34.25
PhaseNetLight	CPU	20	0	91	35.38
EQTransformer-NC	CPU	20	0	228	35.45
EQTransformer	CPU	20	0	137	38.42
EQTransformer	GPU	20	1	22	52.48
EQTransformer-NC	GPU	20	1	22	53.31
EQTransformer-NC	GPU	20	2	44	54.17
EQTransformer	GPU	20	2	44	55.88
PhaseNetLight	GPU	20	1	22	55.95
PhaseNet	GPU	20	2	44	56.27
PhaseNet	GPU	20	1	22	59.32
PhaseNetLight	GPU	20	2	44	67.61
EQCCT	CPU	20	0	137	76.07
EQCCT	GPU	20	2	24	96.10
EQCCT	GPU	20	1	12	127.35

Table 4. Best successful Model-Actor total time at 228 stations, sorted by fastest time. CPU rows are minima over Ray CPUs tested; GPU rows report the best one-GPU and two-GPU Model-Actor totals separately. Setup OH is actor-creation (or equivalent) setup time as a percentage of total Model-Actor trial time.

Model	Device	CPUs	GPUs	Actors	Setup (s)	Pick (s)	Total (s)	Setup OH
PhaseNet	GPU	20	1	22	6.70	4.27	10.97	61.0%
PhaseNetLight	CPU	20	0	46	7.04	4.28	11.47	61.4%
PhaseNet	CPU	20	0	46	7.21	4.16	11.48	62.8%
PhaseNetLight	GPU	20	1	22	6.96	4.56	11.53	60.4%
EQTransformer	CPU	20	0	46	6.92	4.70	11.63	59.5%
EQT-NC	GPU	20	1	22	6.91	5.19	12.11	57.1%
EQCCT	GPU	14	1	12	4.64	7.82	12.47	37.2%
EQTransformer	GPU	20	1	22	7.12	6.92	14.05	50.7%
EQCCT	GPU	17	2	24	7.95	9.33	17.29	46.0%
PhaseNet	GPU	20	2	44	10.88	6.60	17.49	62.2%
EQT-NC	CPU	20	0	46	12.88	4.70	17.74	72.6%
PhaseNetLight	GPU	20	2	44	10.83	7.02	17.86	60.6%
EQT-NC	GPU	20	2	44	10.75	7.30	18.06	59.5%
EQTransformer	GPU	20	2	44	10.76	8.16	18.93	56.9%
EQCCT	CPU	14	0	46	12.73	12.23	25.01	50.9%

Table 5. Memory utilization for each Model-Actor row in Table 4. All memory values are in MB.

Model	Device	CPUs	GPUs	Actors	Req. RAM	Tree RAM	Req. VRAM	Tree VRAM	Peak RAM
PhaseNet	GPU	20	1	22	58,564	32,813	36,344	11,580	782
PhaseNetLight	CPU	20	0	46	105,524	49,017	0	0	273
PhaseNet	CPU	20	0	46	105,524	51,694	0	0	274
PhaseNetLight	GPU	20	1	22	58,366	39,068	36,344	11,580	280
EQTransformer	CPU	20	0	46	106,398	51,012	0	0	258
EQT-NC	GPU	20	1	22	61,798	43,578	37,004	12,273	244
EQCCT	GPU	14	1	12	49,236	15,363	34,608	40,316	456
EQTransformer	GPU	20	1	22	61,446	42,982	36,960	12,226	275
EQCCT	GPU	17	2	24	98,472	18,506	69,216	43,675	958
PhaseNet	GPU	20	2	44	117,128	59,490	72,688	526	1,084
EQT-NC	CPU	20	0	46	106,536	56,537	0	0	200
PhaseNetLight	GPU	20	2	44	116,732	72,936	72,688	23,161	383
EQT-NC	GPU	20	2	44	123,596	82,457	74,008	24,545	378
EQTransformer	GPU	20	2	44	122,892	80,613	73,920	24,453	398
EQCCT	CPU	14	0	46	115,920	48,701	0	0	202